Medical Concierge — Developer Guide

An educational walkthrough for a new developer: what this application is, exactly how it works end to end, and how Docker is used to run it.

---

1. What you're looking at

Medical Concierge is a **single-user, self-hosted** web application for a patient juggling multiple doctors who don't coordinate with each other. The user photographs pill bottles, uploads visit-note PDFs, or snaps pictures of handwritten prescriptions. The app:

1. **Reads** those documents with a vision-capable LLM (Claude),
2. **Normalizes** every medicine/supplement name against RxNorm, the U.S. National Library of Medicine's drug vocabulary,
3. **Screens** the combined record set for drug–drug interactions, drug–supplement/vitamin/mineral interactions, duplicate therapy, and nutrient depletions,
4. **Scores its own confidence** at every step and flags anything shaky, and
5. **Exports** everything — including a clinician-readable PDF the patient can hand to a doctor.

Two framing principles shape almost every design decision in the codebase:

- **This is not a diagnostic device.** Every output is phrased as "worth discussing with your doctor/pharmacist," never as an instruction. You'll see this discipline in prompt text, UI copy, and PDF wording alike.
- **The LLM is never the final authority on a drug's identity.** The model reads documents; RxNorm decides what a name actually refers to. Anything that can't be confidently normalized is kept but flagged, never guessed.

Related documents: `ARCHITECTURE.md` (the long-term system plan this MVP fits into), `MVP_INGESTION_AGENTS.md` (why the ingestion design is what it is), `WINDOWS_SETUP.md` (end-user setup), `../deploy/README.md` (deployment).

---

2. The 10,000-foot view

```
Browser (static/index.html — one self-contained file)
   |  fetch()
   v
FastAPI app (app/main.py + app/api/routes.py)
   |
   |-- POST /api/ingest/{medicine|supplement}
   |       |
   |       v
   |   agents/ ---------> ingestion/file_loader.py    (PDF -> page images)
   |       |             ingestion/multimodal_extractor.py (Claude vision,
   |       |                                                forced tool call)
   |       |------------> normalization/rxnorm_client.py   (RxNorm API)
   |       |             normalization/supplement_terms.py (local fallback)
   |       v
   |   storage/store.py  (SQLite)
   |
   |-- GET /api/findings --> interactions/engine.py + knowledge_base.py
   |                          (deterministic screening, no LLM, no network)
   |
   '-- GET /api/export ----> export/pdf_report.py (PyMuPDF) | CSV | JSON
```

Tech stack:

Layer	Choice	Why
Backend	Python 3.11 + FastAPI	async I/O for API calls; best medical/LLM library ecosystem
Validation	Pydantic v2	every boundary (LLM output, API, storage) is schema-validated
Document reading	Claude vision + forced tool call	no custom OCR pipeline; schema-valid output by construction
Drug vocabulary	RxNorm REST API (free, no key)	the industry-standard normalizer; built for fuzzy input
PDF read/write	PyMuPDF (`fitz`)	one library both rasterizes incoming PDFs and writes the export PDF
Storage	SQLite, single file	single user, zero ops; upgrade path is Postgres (see ARCHITECTURE.md)
Frontend	one static HTML file, vanilla JS	no build step, no CDNs, works offline, trivially auditable
Deployment	Windows .bat (desktop) or Docker Compose + Caddy (server)	see §9

There is deliberately **no framework magic**: no ORM, no frontend framework, no agent orchestration library. Every moving part is a plain function you can read top to bottom.

---

3. Repository tour

```
MedicalConcierge/
├── Start-MedicalConcierge.bat   # double-click Windows launcher (see §9.1)
├── docker-compose.yml           # server deployment: app + Caddy proxy (§9.2)
├── deploy/
│   ├── Caddyfile                # TLS-terminating reverse proxy config
│   └── README.md                # deployment instructions
├── docs/                        # you are here
└── backend/
    ├── Dockerfile               # container image for the app (§9.2)
    ├── requirements.txt         # runtime deps; -dev adds pytest
    ├── static/index.html        # the entire frontend
    ├── app/
    │   ├── main.py              # FastAPI wiring: /api router + static mount
    │   ├── config.py            # Settings (pydantic-settings, reads .env)
    │   ├── schemas.py           # ALL shared data models — start reading here
    │   ├── api/routes.py        # every HTTP endpoint
    │   ├── ingestion/           # file -> images -> extracted items
    │   ├── normalization/       # names -> RxNorm codes
    │   ├── agents/              # pipeline orchestration per record kind
    │   ├── interactions/        # screening rules + engine
    │   ├── export/              # clinician PDF builder
    │   └── storage/             # SQLite persistence
    └── tests/                   # all offline; LLM + network mocked
```

Reading order for a new developer: `schemas.py` → `agents/common.py` → `ingestion/` → `normalization/` → `interactions/engine.py` → `api/routes.py` → `static/index.html`. Roughly 1,600 lines of Python total — an afternoon.

---

4. The data model (`app/schemas.py`)

Four models matter; everything else in the system is a function between them.

`ExtractedItem` — one medication/supplement mention *as read off a document*, before anyone decides what it actually is. Fields: `raw_text` (verbatim, so a human can audit the reading), `name_as_written`, `dosage`, `form`, `route`, `frequency`, `prescriber_or_source`, `date_documented`, `source_type`, `extraction_confidence` (0–1, set by the model against a rubric), and `ambiguities` (a list of plain-language notes like *"Dose digit unclear — could be 10 or 40"*).

`RxNormMatch` — what normalization concluded: `rxcui` (RxNorm's concept ID), `canonical_name`, `match_score` (RxNorm's 0–100), a derived `normalization_confidence` (0–1), and `source` (`"rxnorm"` or `"local_supplement_table"`).

`NormalizedRecord` — the persisted unit: an `ExtractedItem` + an `RxNormMatch` + derived fields. The important logic is in `NormalizedRecord.build()`:

```
overall = extracted.extraction_confidence * normalization.normalization_confidence
needs_review = overall < review_threshold   # default 0.6
```

The multiplication is deliberate: a perfectly-read name that failed to normalize, and a beautifully-normalized guess at illegible handwriting, should **both** score low. Either failure alone is disqualifying, so the combiner is a product, not an average.

`Finding` — one screening result (interaction, duplicate, or depletion): severity (`major` / `moderate` / `info`), category, title, the involved record IDs and display names, a plain-language `explanation`, a discussion-framed `recommendation`, an `evidence_note`, and `reading_confidence` — the *minimum* `overall_confidence` among involved records, because a finding is only as trustworthy as the shakiest reading underneath it.

---

5. Life of a document

The best way to understand the app is to follow one upload end to end. Say the user photographs a pill bottle and clicks **Read this document** with "Medicine" selected.

5.1 Upload (`static/index.html` → `POST /api/ingest/medicine`)

The frontend sends a `multipart/form-data` request with the file and a `source_type` query parameter (`bottle_label`, `handwritten_note`, ...). That hint is passed through to the extraction prompt so the model can calibrate its own confidence reporting.

5.2 File → images (`ingestion/file_loader.py`)

`load_as_images()` returns a list of `(image_bytes, mime_type)` tuples:

- Small images (`.jpg/.jpeg/.png/.webp`) pass through byte-for-byte.
- **Oversized images are downscaled and re-encoded.** The vision API rejects individual images over ~5 MB and internally downscales anything past ~1568 px on the long side — so a 12 MP phone photo sent raw is both a "file too large" error and wasted upload. Anything beyond 1568 px or 1.5 MB is rendered down to a 1568 px long-side JPEG (quality 80) with PyMuPDF. No accuracy is lost; the API would have discarded those pixels.
- **PDFs are always rasterized** to one JPEG per page, at the configured DPI but never beyond the same 1568 px cap, with a 50-page guardrail.

The API layer above this adds a 30 MB request cap with a friendly message and converts vision-API failures into clean HTTP 502s instead of raw 500s.

Why rasterize even text-based PDFs? Real-world medical PDFs are frequently scans or faxes with no text layer. A "try the text layer, fall back to images" branch would double the code paths for zero reliability gain — and a printed page is a trivially easy image for a vision model. One uniform path.

5.3 Vision extraction (`ingestion/multimodal_extractor.py`)

Page images go to Claude alongside a system prompt that makes the model a "meticulous medical records transcriptionist." Short documents fit one request; long ones are **batched** — pages are grouped so no request exceeds ~15 MB of raw image bytes (the API caps total request size at ~32 MB, and base64 inflates by 4/3), each batch is labeled "part N of M of the same document," and the extracted items are merged in order. Three techniques here are the heart of the ingestion design:

1. **Forced tool call.** The request defines a `record_extraction` tool with a full JSON Schema and sets `tool_choice={"type": "tool", ...}`. The model *cannot* reply with prose; the API guarantees a schema-shaped response. There is no "parse the model's JSON and hope" step anywhere. The result is then re-validated through Pydantic (`ExtractedItem`) as a second belt-and-suspenders layer.
2. **An explicit confidence rubric** in the prompt (0.9+ only for clearly printed text; 0.3–0.59 for partially legible handwriting; etc.). Without anchoring, model self-reported confidence is noise; with it, the numbers are consistent enough to drive the `needs_review` threshold.
3. **Anti-guessing instructions.** The model must extract crossed-out or ambiguous items *and say so* in `ambiguities` rather than silently resolving or omitting them, must report verbatim `raw_text` for auditing, and must never infer a drug identity the visible text doesn't support.

5.4 Normalization (`normalization/`)

Each extracted name goes to RxNorm's `approximateTerm.json` endpoint — purpose-built for misspellings, brand names, and OCR noise. The client takes the highest-scoring candidate, resolves its canonical name via a second `rxcui/{id}/property.json` call, and converts RxNorm's 0–100 score into `normalization_confidence`.

So *"Tylenol"* — however the doctor scrawled it — becomes `rxcui=161, canonical_name="acetaminophen"`. That's what makes downstream screening possible: every rule matches against canonical vocabulary, not against whatever was written.

Supplements get one extra step (`agents/supplement_agent.py`): if RxNorm's best score is below 50, a curated ~40-entry local synonym table (`supplement_terms.py`) is tried — RxNorm has vitamins but not most herbal products. Table matches get a fixed, honest `normalization_confidence` of 0.75 and `source="local_supplement_table"`. The production plan replaces this table with TRC NatMed Pro (licensed) behind the same interface.

If neither source matches, the record is stored anyway with confidence 0 — **flagged `unnormalized`, never silently dropped, never guessed.**

5.5 Orchestration (`agents/`)

`agents/common.py:process_document()` is the whole pipeline in ~20 lines: load images → extract → for each item, normalize → `NormalizedRecord.build()`. The medicine and supplement agents are thin wrappers that inject their own normalization strategy as an async callable. That's the entire "agent framework" — dependency injection of one function.

5.6 Storage (`storage/store.py`)

SQLite, one table. Each record's full Pydantic JSON goes into a `payload` column, with a few extracted columns (`kind`, `needs_review`, `overall_confidence`, timestamps) for filtering. Reads are `model_validate(json.loads(...))` — the schema is the source of truth, not the table shape. MVP-grade and explicitly unencrypted; ARCHITECTURE.md §5 covers what must change before real PHI on a shared machine.

5.7 Screening (`interactions/`) — runs after *every* change

The UI refetches `GET /api/findings` after each upload. The engine (`engine.py:evaluate()`) is a **pure function** of the record list: deterministic, no LLM, no network, instantly recomputable, trivially testable. It runs three passes:

- **Interaction rules** (`knowledge_base.py:INTERACTION_RULES`): each rule has two term-lists (e.g., ACE inhibitors × potassium). If any record matches side A and a *different* record matches side B, a Finding is emitted. Matching uses word-boundary regex against the canonical name (fallback: as-written), so the term `iron` can never match "Ironwood Herbal Blend".
- **Duplicate detection**: two different members of the same therapeutic class (two NSAIDs → classic two-doctors-not-talking), or the same ingredient appearing in multiple source documents.
- **Depletion rules**: e.g. metformin depletes B12. Note the tone switch: if the user *already* takes the depleted nutrient, the recommendation becomes "mention it so levels get checked" instead of "consider adding" — the engine checks the supplement list before recommending.

Findings sort most-severe first, then weakest-reading-confidence first within a band, so the user verifies shaky records early. Every rule in the knowledge base is a pharmacy-handout-level documented fact, and the UI/PDF label the whole feature as "a starter screen, not a complete check."

5.8 Render

The frontend rerenders the record list (confidence badges, "check this" flags, ambiguity callouts) and the warnings panel (severity-colored cards with explanation + 💡 recommendation + confidence caveat). All rendering is ~100 lines of vanilla JS operating on the JSON the API returns.

---

6. The API surface (`app/api/routes.py`)

Endpoint	What it does
`GET /api/health`	`{ok, anthropic_key_configured}` — drives the UI's "Setup needed" banner
`POST /api/ingest/medicine`	multipart upload → full pipeline → stored records returned
`POST /api/ingest/supplement`	same, with the supplement normalization strategy
`GET /api/records?kind=`	all stored records, newest first
`GET /api/findings`	screening results, recomputed on the fly
`GET /api/export?format=json|csv|pdf`	full export; PDF is the clinician summary
`GET /`	serves `static/index.html`

There is no auth: the server binds to `127.0.0.1` in desktop mode, and in Docker mode is reachable only through the reverse proxy on a LAN/VPN. Anything beyond that (passkeys/WebAuthn) is future work per ARCHITECTURE.md.

---

7. Exports (`app/export/pdf_report.py`)

CSV and JSON are straightforward serializations in `routes.py`. The PDF is the interesting one: it's built with PyMuPDF's low-level text API through a small `_Writer` class that owns a y-cursor, measures text with `fitz.get_text_length()` for word-wrapping, and page-breaks automatically.

The layout order is intentional: **Potential Interactions & Recommendations come first** (a clinician should see warnings before the med list), then Medications, then Supplements, alphabetically. Low-confidence entries carry a red `** PLEASE CONFIRM **` marker with their ambiguity notes, and each finding block shows severity, involved records, mechanism, suggested next step, and the confidence caveat. Every page footer restates "not medical advice."

---

8. Configuration (`app/config.py`)

`pydantic-settings` reads environment variables and `backend/.env` (written by the Windows launcher's first-run prompt, or created by hand — one `ANTHROPIC_API_KEY=...` line is enough; everything else has defaults):

Variable	Default	Meaning
`ANTHROPIC_API_KEY`	*(empty)*	the one required secret; empty → UI shows setup banner
`EXTRACTION_MODEL`	`claude-sonnet-5`	vision model for extraction
`RXNORM_BASE_URL`	`https://rxnav.nlm.nih.gov/REST`	swappable for tests/mirrors
`DB_PATH`	`./medconcierge.sqlite3`	Docker sets `/data/medconcierge.sqlite3`
`REVIEW_CONFIDENCE_THRESHOLD`	`0.6`	below this, records are flagged `needs_review`
`PDF_RENDER_DPI`	`200`	rasterization resolution for ingested PDFs (output is still capped at 1568 px long-side for the vision API)

---

9. How the app is run — two deployment modes

9.1 Desktop mode (`Start-MedicalConcierge.bat`)

For the primary user — a non-technical patient on Windows — Docker is too much to ask. The launcher is a plain batch file that: finds Python (with plain-English install guidance if missing) → creates `.venv` on first run → installs `requirements.txt` (re-running only when the file changes, tracked by comparing a copied marker) → prompts for the API key once and writes `backend\.env` → starts `uvicorn` bound to `127.0.0.1:8000` → opens the browser. Closing the console window stops the server; the SQLite file persists. No command line knowledge needed.

9.2 Server mode: how Docker is used

For an always-on home server/NAS/VPS, the repo ships a two-container Docker Compose stack. **Why containers here?** Reproducibility (pinned Python and dependencies independent of the host), painless updates (`git pull && docker compose build && up -d` with data untouched), process supervision (`restart: unless-stopped`), and network isolation (the app container is never directly exposed).

The image (`backend/Dockerfile`)

```
FROM python:3.11-slim                  # small official base
RUN groupadd -r medconcierge && useradd -r -g medconcierge medconcierge
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt   # own layer: cached
COPY app ./app                         # unless requirements change
COPY static ./static
RUN mkdir -p /data && chown -R medconcierge:medconcierge /data /app
VOLUME ["/data"]                       # persistent data lives here
ENV DB_PATH=/data/medconcierge.sqlite3
EXPOSE 8000
USER medconcierge                      # never run as root
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Things worth teaching from these few lines:

- **Layer-cache ordering**: `requirements.txt` is copied and installed *before* the application code, so day-to-day code changes rebuild in seconds — the dependency layer only rebuilds when requirements change. (`.dockerignore` keeps `.venv`, tests, `.env`, and caches out of the build context, which also prevents the real API key from ever being baked into an image.)
- **Non-root user**: a container escape or app compromise lands in an unprivileged account.
- **`0.0.0.0` inside the container is safe** because of the network design below: the port is `expose`d to the internal Docker network only, never `ports`-published to the host.
- **State separation**: the image is disposable; everything worth keeping is under `/data`, a declared volume.

The stack (`docker-compose.yml`)

```
services:
  app:                     # FastAPI, built from backend/Dockerfile
    env_file: ./backend/.env
    volumes: [medconcierge-data:/data]
    expose: ["8000"]       # internal network only — no host port
    healthcheck:           # GET /api/records must succeed
    networks: [internal]
  proxy:                   # Caddy, the ONLY container with host ports
    image: caddy:2-alpine
    depends_on: {app: {condition: service_healthy}}
    ports: ["443:443", "80:80"]
    volumes: [./deploy/Caddyfile:/etc/caddy/Caddyfile:ro, caddy-data:/data, ...]
    networks: [internal]
```

The two containers share a private `internal` network. The proxy resolves the app by its service name — that's the `reverse_proxy app:8000` line in the Caddyfile; Docker's embedded DNS maps `app` to the container's internal IP. The `depends_on … service_healthy` gate means Caddy won't start routing until the app's healthcheck (an actual API call, not just "process exists") passes.

Three named volumes survive rebuilds: `medconcierge-data` (the SQLite database — the only thing that truly matters; the backup command in `deploy/README.md` tars exactly this), plus Caddy's cert/config state.

The proxy (`deploy/Caddyfile`)

Caddy terminates TLS and adds security headers. Because this system is LAN/VPN-only *by design* (no public domain), it uses `tls internal` — Caddy mints its own CA and issues itself a cert; the user trusts that root cert once per device (or skips Caddy entirely and uses `tailscale serve`, which brings its own certs — both paths are documented in `deploy/README.md`). Port 80 exists only to redirect to 443.

The deliberate security posture, from ARCHITECTURE.md §7: **never expose this to the public internet.** Remote/phone access goes through a personal VPN (Tailscale/WireGuard). The only outbound traffic is the Claude API (document images) and RxNorm (drug names only) — and that scope is documented to the user rather than pretending the system is air-gapped.

Day-2 operations

```
docker compose build && docker compose up -d   # deploy / update
docker compose logs -f app                     # watch the app
docker compose ps                              # health status
# backup (see deploy/README.md):
docker run --rm -v medconcierge-data:/data -v "$PWD":/backup alpine \
  tar czf /backup/medconcierge-data-$(date +%F).tar.gz -C /data .
```

---

10. Testing (`backend/tests/`)

```
cd backend && pip install -r requirements-dev.txt && pytest
```

All tests run **offline with no API key** — that's a design requirement, not an accident. The seams that make it possible:

- The Claude call is one function (`extract_records`); agent tests monkeypatch it at its import site in `agents/common.py` and feed hand-built `ExtractedItem`s.
- The RxNorm client's HTTP layer is `httpx.AsyncClient.get`; tests monkeypatch that with canned JSON responses.
- The screening engine and PDF builder are pure functions — tests feed records in and assert on findings out (or extract text back out of the generated PDF with PyMuPDF and assert on it).

What the suite covers: confidence math and thresholds, RxNorm candidate selection and the empty/failure paths, the supplement fallback decision, both agents end-to-end (with mocks), ten screening-engine behaviors (severity, categories, duplicate detection, word-boundary matching, depletion tone switching, sort order, confidence propagation), and PDF content/pagination. What it deliberately can't cover: real Claude reading accuracy and live RxNorm behavior — those need a manual smoke test with a real key.

---

11. Extending the system

Add an interaction rule — append an `InteractionRule` to `INTERACTION_RULES` in `knowledge_base.py` (or a `DepletionRule`, or a class in `DUPLICATE_CLASSES`). Terms are matched with word boundaries against canonical names; add a test in `test_interaction_engine.py`. No other file changes — the engine, API, UI, and PDF all pick it up.

Swap in a real interaction database (openFDA, NatMed Pro) — implement a provider that yields the same rule/finding shapes and merge its output in `engine.evaluate()`. The UI and PDF render `Finding`s; they don't care where rules came from. Keep the curated list as the offline fallback.

Add a document type (e.g. lab results) — new extractor prompt + schema in `ingestion/`, a normalization target (LOINC for labs), and a new `RecordKind`. `agents/common.py` shows the pattern to copy.

The bigger roadmap — diagnosis gap analysis, supplement evidence grading, FHIR export, encryption at rest, WebAuthn — is sequenced with rationale in `ARCHITECTURE.md` §8.

---

12. Privacy model (know this before touching anything)

Data that leaves the machine, exhaustively: (1) document images/text → the Claude API for extraction; (2) drug/supplement *names only* → RxNorm. Nothing else. No analytics, no telemetry, no cloud storage. All records live in one local SQLite file the user can copy, back up, or delete. When you add features, preserve this property or document the change loudly — it is the product's core promise.

Appendix A — Glossary

Term	Meaning
RxNorm	NLM's standard vocabulary of drugs; the universal translator between "Tylenol", "APAP", and "acetaminophen"
RxCUI	RxNorm Concept Unique Identifier — the stable ID for one drug concept
Normalization	mapping free-text names onto standard codes so different spellings unify
Forced tool call	an Anthropic API mode where the model must respond by "calling" a developer-defined, JSON-Schema-validated function — structured output by construction
Rasterization	rendering a PDF page into a bitmap image (here: 200 DPI PNG)
`needs_review`	record flag set when `overall_confidence < 0.6`; surfaces as "check this" in the UI
Finding	one screening result: interaction, duplicate therapy, or nutrient depletion
Reverse proxy	the Caddy container: terminates TLS, adds headers, forwards to the app over the internal Docker network
Named volume	Docker-managed persistent storage that outlives containers; where the SQLite DB lives in server mode
`tls internal`	Caddy's self-managed private CA — right for LAN/VPN services that will never have a public domain

Appendix B — Quick-start for development

```
git clone <repo> && cd MedicalConcierge/backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env   # only needed to test real ingestion
pytest                          # 22 tests, all offline
uvicorn app.main:app --reload   # http://localhost:8000
```

To regenerate this guide's PDF: `python docs/build_dev_guide.py` (requires `pip install markdown playwright` and a Chromium — see the script's docstring).